

同济大学计算机系

计算机组成原理课程实验报告



学 号 2351520

姓 名 毛星博

专 业 计算机科学与技术

授课老师 陈永生

32 位有符号除法器

一、实验内容

有符号除法器 带符号除法器功能为：将两个 32 位有符号数相除，得到一个 32 位商和余数，基本和无符号除法器类似，注意余数符号与被除数符号相同。

二、硬件逻辑图：无

三、模块建模

```
`timescale 1ns / 1ps

module DIV(
    input [31:0] dividend,    // 被除数
    input [31:0] divisor,     // 除数
    input start,              // 启动除法运算
    input clock,
    input reset,
    output [31:0] q,          // 商
    output [31:0] r,          // 余数
    output reg busy           // 除法器忙标志位
);

    wire ready;
    reg [5:0] count;          // 计数器，用于 32 位运算需要 6 位
    reg [31:0] reg_q;         // 商寄存器
    reg [31:0] reg_r;         // 余数寄存器
    reg [31:0] reg_b;         // 除数寄存器
    reg busy2, r_sign;        // busy2 用于产生 ready 信号，r_sign 表示余数符号
    reg q_sign;               // 商的符号
    reg dividend_sign;        // 被除数的符号

    // ready 信号生成
    assign ready = ~busy & busy2;

    // 加减法器，根据 r_sign 选择加法或减法
    wire [32:0] sub_add = r_sign ?
        ({reg_r, reg_q[31]} + {1'b0, reg_b}) :
        ({reg_r, reg_q[31]} - {1'b0, reg_b});

    // 余数恢复和最终结果计算
    wire need_restore = r_sign;
```

```

wire [31:0] restored_r = need_restore ?
    reg_r + reg_b :
    reg_r;

// 最终输出，根据符号确定最终的余数和商
assign r = dividend_sign ? -restored_r : restored_r;
assign q = q_sign ? -reg_q : reg_q;

// 主状态控制
always @(posedge clock or posedge reset) begin
    if (reset == 1) begin
        // 复位状态
        count <= 6'b0;
        busy <= 0;
        busy2 <= 0;
    end else begin
        busy2 <= busy;
        if (start) begin
            // 开始除法运算，初始化各寄存器
            // 保存符号信息
            dividend_sign <= dividend[31];
            q_sign <= dividend[31] ^ divisor[31];

            // 初始化除法运算
            reg_r <= 32'b0;
            r_sign <= 0;
            reg_q <= dividend[31] ? -dividend : dividend;
            reg_b <= divisor[31] ? -divisor : divisor;
            count <= 6'b0;
            busy <= 1'b1;
        end else if (busy) begin
            // 循环操作
            reg_r <= sub_add[31:0];    // 更新部分余数
            r_sign <= sub_add[32];    // 更新符号位，决定下次是加还是减
            reg_q <= {reg_q[30:0], ~sub_add[32]}; // 更新商
            count <= count + 6'b1;    // 计数器加 1
            if (count == 6'h1F)       // 32 位运算完成
                busy <= 0;           // 清除忙标志
        end
    end
end

endmodule

```

四、测试模块建模

```
`timescale 1ns / 1ps

module DIV_tb;
    // 定义信号
    reg [31:0] dividend;
    reg [31:0] divisor;
    reg start;
    reg clock;
    reg reset;
    wire [31:0] q;
    wire [31:0] r;
    wire busy;

    // 实例化被测试模块
    DIV uut (
        .dividend(dividend),
        .divisor(divisor),
        .start(start),
        .clock(clock),
        .reset(reset),
        .q(q),
        .r(r),
        .busy(busy)
    );

    // 生成时钟信号
    initial begin
        clock = 0;
        forever #5 clock = ~clock;
    end

    // 测试用例
    initial begin
        // 初始化信号
        reset = 1;
        start = 0;
        dividend = 0;
        divisor = 0;
        #10;
```

```

reset = 0;
#10;

// 测试用例 1: 正数除以正数 ( $50 / 3 = 16 \dots 2$ )
dividend = 32'd50;
divisor = 32'd3;
start = 1;
#10;
start = 0;
wait(!busy);
#10;

// 测试用例 2: 正数除以负数 ( $50 / -3 = -16 \dots 2$ )
dividend = 32'd50;
divisor = -32'd3;
start = 1;
#10;
start = 0;
wait(!busy);
#10;

// 测试用例 3: 负数除以正数 ( $-50 / 3 = -16 \dots -2$ )
dividend = -32'd50;
divisor = 32'd3;
start = 1;
#10;
start = 0;
wait(!busy);
#10;

// 测试用例 4: 负数除以负数 ( $-50 / -3 = 16 \dots -2$ )
dividend = -32'd50;
divisor = -32'd3;
start = 1;
#10;
start = 0;
wait(!busy);
#10;

// 测试用例 5: 除数为 1 的情况
dividend = 32'd100;
divisor = 32'd1;
start = 1;
#10;

```

```
start = 0;  
wait(!busy);  
#10;
```

```
end
```

```
endmodule
```

五、实验结果

Modelsim 仿真图

